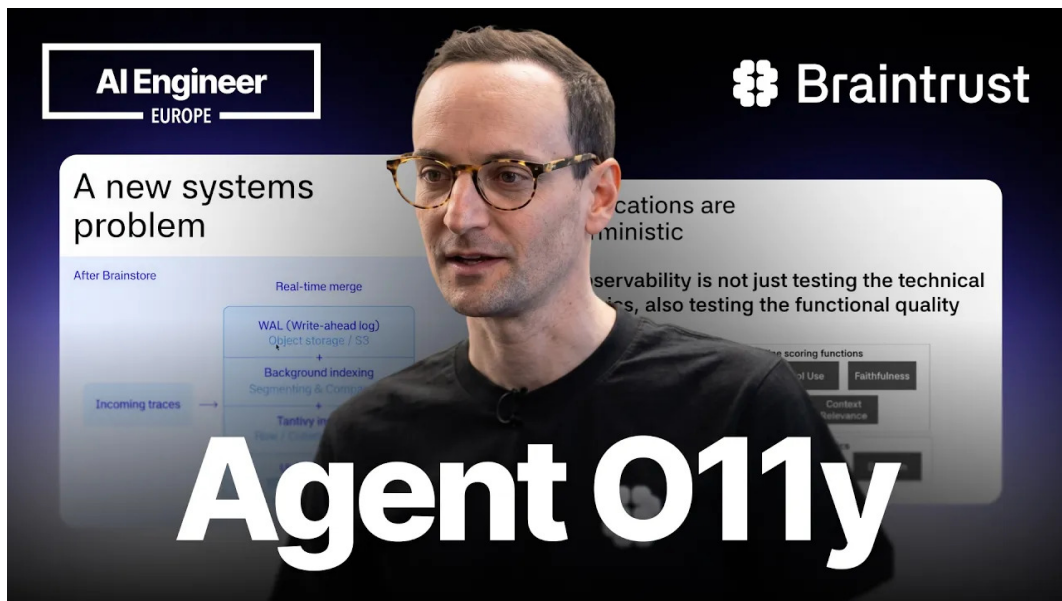


视频课程笔记

# How agent o11y differs from traditional o11y | Phil Hetzel, Braintrust

AI Engineer & Codex

2026-05-30



视频频道: AI Engineer

发布日期: 2026-05-28

视频时长: 20:43

视频链接: <https://www.youtube.com/watch?v=XBaznoTRDFI>

资料说明: 本笔记以 YouTube 英文自动字幕、视频画面、封面图与本地元数据为基础整理。自动字幕中的明显识别错误已按上下文修正, 公司名统一写作 Braintrust。

# 目录

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>1 问题设定：为什么 Agent o11y 会成为单独议题</b>                  | <b>3</b>  |
| 1.1 议题入口：看起来同属“监控”，实际问题集已经换了                          | 3         |
| 1.2 讲者与 Braintrust 背景：从 POC 到生产化的断点                   | 4         |
| 1.3 Agent quality 的两条线：生产信任与实验信心                      | 4         |
| 1.4 为什么 agent observability 会分离出来                     | 5         |
| 1.5 本章小结                                              | 5         |
| <b>2 传统可观测性的边界：uptime、性能、metrics 与 trace/span</b>     | <b>5</b>  |
| 2.1 Uptime 与 technical performance：系统是否在线，体验是否达标      | 5         |
| 2.2 Metrics：latency、duration、4xx/5xx error count      | 6         |
| 2.3 Trace 与 span：一次 workflow 与其中一步                    | 6         |
| 2.4 Grafana 与 Datadog：成熟工具适合成熟问题                      | 8         |
| 2.5 为什么这些 building blocks 仍会被 Agent o11y 继承           | 8         |
| 2.6 本章小结                                              | 9         |
| <b>3 第一差异：Agent 的非确定性与质量指标扩张</b>                      | <b>9</b>  |
| 3.1 同一套 building blocks，换了一类行为对象                      | 9         |
| 3.2 确定性控制流与 LLM 高变化行为                                 | 9         |
| 3.3 从“快不快”扩展到“答得好不好”                                  | 10        |
| 3.4 Groundedness：回答是否扎根于证据                            | 11        |
| 3.5 Tool-use correctness：工具调用是否合乎任务                   | 12        |
| 3.6 Brand alignment：系统提示与品牌边界是否被执行                    | 12        |
| 3.7 本章小结                                              | 12        |
| <b>4 第二差异：Agent trace 的形状、体积与实时压力</b>                 | <b>12</b> |
| 4.1 质量指标扩张会反向改变 trace 的数据形状                           | 12        |
| 4.2 半结构化外壳与非结构化文本内核                                   | 13        |
| 4.3 从 KB 级记录到 GB 级 trace、20MB span                    | 13        |
| 4.4 实时压力：ingest、process、use 必须同时成立                    | 14        |
| 4.5 Braintrust trace UI：model call、tool call 与长文本同屏出现 | 14        |
| 4.6 系统挑战图中的量级对比                                       | 16        |
| 4.7 本章小结                                              | 16        |
| <b>5 数据系统方案：WAL、索引、Tantivy 与 SQL 查询</b>               | <b>17</b> |
| 5.1 两种读模式：实时 UI 与 CLI/SQL 分析                          | 18        |
| 5.2 Write-ahead log：先让 trace 进入可见轨道                   | 18        |
| 5.3 过滤与分析索引：给结构化字段修目录                                 | 18        |
| 5.4 Tantivy 全文索引：为什么“Amazon in trace”是中心例子            | 19        |
| 5.5 SQL-like 统一查询层：把多种能力收束成一种入口                       | 20        |
| 5.6 本章小结                                              | 20        |
| <b>6 第三个差异：使用者从工程师扩展到领域专家</b>                         | <b>21</b> |
| 6.1 传统 observability 的典型用户                            | 22        |
| 6.2 SME 为什么能直接参与 Agent 改进                             | 22        |
| 6.3 Prompt：自然语言作为贡献接口                                 | 23        |
| 6.4 Observability 与 evals：同一个系统问题的两种运行形态              | 23        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 6.5      | 本章小结                                                   | 24        |
| <b>7</b> | <b>下一阶段：从 trace 到主题、意图、情绪与问题发现</b>                     | <b>24</b> |
| 7.1      | 看似简单的问题，背后需要复杂系统托底                                     | 24        |
| 7.2      | 从 trace 到 embedding：把复杂交互压缩成可计算向量                      | 25        |
| 7.3      | 从 clustering 到 topic modeling：把个体交互组织成群体模式             | 25        |
| 7.4      | intent、sentiment 与 issue discovery：从发生了什么走向为什么重要       | 26        |
| 7.5      | 缩短从生产问题到实验修复的迭代回路                                      | 26        |
| 7.6      | 本章小结                                                   | 27        |
| <b>8</b> | <b>Q&amp;A：功能可观测性、人类标注与闭环优化</b>                        | <b>27</b> |
| 8.1      | functional observability 与 technical observability 的边界 | 27        |
| 8.2      | Human annotation：专家判断必须携带理由                            | 28        |
| 8.3      | 从标注理由到 failure modes，再到 scalable scoring functions     | 28        |
| 8.4      | ClickHouse 历史与文本索引动机                                   | 29        |
| 8.5      | 把生产 trace 加入离线数据集                                      | 29        |
| 8.6      | known unknowns 与 unknown unknowns                      | 30        |
| 8.7      | 本章小结                                                   | 30        |
| <b>9</b> | <b>总结与延伸</b>                                           | <b>30</b> |
| 9.1      | 演讲者的收束观点                                               | 30        |
| 9.2      | 概念压缩：从状态监控到行为诊断                                        | 30        |
| 9.3      | 给工程团队的落地启示                                             | 31        |
| 9.4      | 开放问题                                                   | 31        |
| 9.5      | 拓展阅读                                                   | 31        |

# 1 问题设定：为什么 Agent o11y 会成为单独议题

## 1.1 议题入口：看起来同属“监控”，实际问题集已经换了

Phil Hetzel 的开场并没有把主题放在某个具体产品功能上，他先把问题摆出来：传统 observability 与 agent observability 到底差在哪里。这里的 observability 常被缩写为 o11y，意思是 o 与 y 中间省略了 11 个字母。中文可以译作“可观测性”，但这个词的含义超出“仪表盘上有几条曲线”。更准确地说，它是一套让团队回答系统状态、用户体验、故障原因和改进方向的数据能力。

传统可观测性像一套机器体检系统：测体温、心率、血压，看机器是否在线、请求是否过慢、错误是否增加。Agent 可观测性更像一份带病历、问诊记录、诊断依据和专家复核意见的诊疗系统。读者需要同时看见运行是否健康，也要判断回答是否有依据、工具是否用对、推理路径是否合理、输出是否符合业务标准。

### 本章的关键问题

Agent o11y 成为单独议题，原因在于团队开始追问另一类问题：agent 在生产环境里给出的行为质量能否被信任，改动后的新版本能否更可靠地进入生产。



图 1: 演讲议程：从 Braintrust 背景进入传统可观测性与 Agent 可观测性的差异，最后讨论下一阶段方向。<sup>1</sup>

<sup>1</sup>视频画面时间区间：00:00:28.96–00:00:49.15。

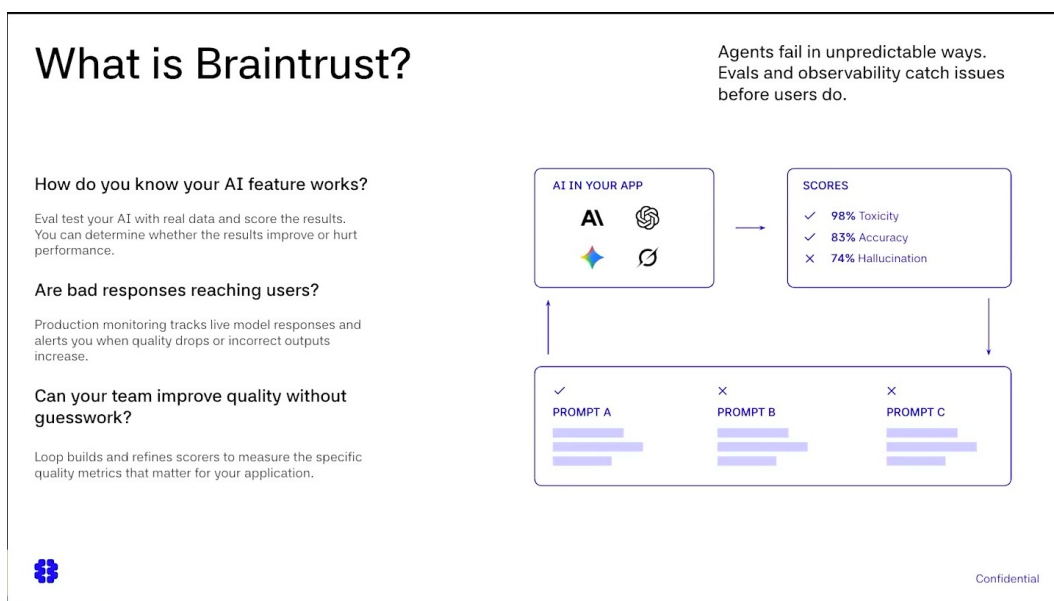


图 2: Braintrust 将 Agent 质量拆成可评分、可监控、可迭代的闭环：应用输出进入评分，再反馈到 prompt 与实验。<sup>2</sup>

## 1.2 讲者与 Braintrust 背景：从 POC 到生产化的断点

讲者 Phil Hetzel 在 Braintrust 负责 solutions engineering。这个角色的核心工作，是帮助客户尽快从平台中得到真实价值。这里的“价值”带有工程落地含义：目标包括把演示做得漂亮、把 agent 放进生产环境，并持续判断它是否在正确工作。

字幕中出现过公司名误识别，正文统一使用 Braintrust。Hetzel 在加入 Braintrust 之前做过 12 年咨询与系统实施，也曾负责 Slalom Consulting 的全球 Databricks 实践。他观察到一个反复出现的企业 AI 痛点：很多客户很擅长做生成式 AI 的 proof of concept (POC，概念验证)，却很难把这些 POC 可靠地带到生产环境。POC 像实验室里跑通的一次化学反应，生产化要求它能在真实流量、真实用户、真实失败条件下稳定运行。

### POC 与生产化的差别

POC 关注“能不能做出来”，生产化关注“能不能长期被信任”。前者通常有受控输入、少量样例和人工兜底；后者面对开放输入、规模化流量、成本约束、错误恢复、审计追踪和版本迭代。

这个背景解释了 Braintrust 的切入点：它把自己定位为 agent quality platform，也就是面向 agent 质量的基础平台。这里的 quality 不能当作抽象赞美词；它指向一组必须被数据化、复盘化、迭代化的问题。

## 1.3 Agent quality 的两条线：生产信任与实验信心

Hetzel 将 Braintrust 对 agent quality 的理解拆成两条线。

- 生产信任：agent 已经上线后，它的表现是否符合团队预期；当用户不断产生新输入时，团队能否继续相信它。
- 实验信心：团队调整 prompt、工具、模型、检索策略或业务规则后，能否知道新版本变好、变坏，还是只是在少数样例上看起来更好。

<sup>2</sup>视频画面时间区间：00:01:46.19–00:02:40.00。

这两条线共同指向一个现实问题：agent 的失败常常不表现为 HTTP 500，也不一定表现为服务宕机。它可能表现为回答缺少依据、遗漏关键约束、调用了错误工具、把检索结果解释错、语气不符合品牌标准，或者在某类用户意图上持续偏离预期。这类失败具有语义性质，必须结合 prompt、上下文、工具调用、模型输出和人工判断一起看。

#### 常见误解：有了 Grafana 或 Datadog 就覆盖了 agent 质量

Grafana、Datadog 这类成熟工具非常适合回答系统运行问题，例如服务是否在线、错误是否升高、接口是否变慢。它们天然擅长处理数值指标和技术事件。Agent 质量问题还包含语义判断：答案是否扎根于证据、是否使用正确工具、是否满足业务定义。只看传统技术指标，很容易得到“系统很健康，用户仍然不满意”的尴尬结论。

### 1.4 为什么 agent observability 会分离出来

演讲的真正起点，是一个客户经常提出的疑问：团队已经有开源工具、Grafana、Datadog，甚至已经有现成合同和实施体系，为什么 agent 场景还要引入一套新的 observability 能力？这个问题很尖锐，因为传统可观测性已经成熟，并且已经证明自己支撑大规模应用。

Hetzel 的回答避开产品宣传，直接从“scope”入手。Scope 可以理解成问题边界：一套系统究竟承诺回答哪些问题、采集哪些数据、支持哪些查询、让哪些角色参与决策。传统 observability 的边界主要围绕技术运行面；agent observability 的边界向行为质量、语义证据、实验迭代和非工程角色协作扩张。

#### 问题边界的变化

传统可观测性问的是：系统是否在线、是否足够快、是否报错。Agent 可观测性继续继承这些问题，同时新增一组质量问题：agent 为什么这样回答、依据是否充足、工具调用是否合理、这个版本是否比上一个版本更值得上线。

### 1.5 本章小结

本章建立了整篇讲稿的出发点。Phil Hetzel 以 Braintrust 的 agent quality 视角切入，指出企业生成式 AI 的关键断点常出现在 POC 到生产化之间。生产环境中的 agent 需要被持续信任，实验环境中的 agent 需要被可靠比较。传统可观测性仍然重要，但它回答的问题主要集中在技术运行面；agent o1ly 的独立性来自新的问题边界：团队必须同时看见运行状态、行为质量、证据链和版本迭代效果。

## 2 传统可观测性的边界：uptime、性能、metrics 与 trace/span

### 2.1 Uptime 与 technical performance：系统是否在线，体验是否达标

在 00:03:16.19–00:04:46.19 这一段，Hetzel 给出传统 observability 的边界：它关注 uptime 和 technical performance。Uptime 指系统是否可用，常见表达是“服务在不在线”“网站能不能打开”“关键接口能不能返回”。Technical performance 指从技术视角看用户体验是否达到预期，例如响应是否过慢、交互是否卡住、某个请求是否失败。

这类观察对象适合被数值化。系统像一条生产线，uptime 对应“机器有没有开着”，latency 对应“每件产品经过这台机器要等多久”，error count 对应“有多少件产品被打回”。当团队面对传统 Web 应用、API 服务、后台任务和基础设施时，这些问题就是可观测性的主战场。

## Availability、uptime 与用户体验

Availability 通常表示可用性，uptime 是它最直观的运行时间表达。一个系统可以 99.9% 在线，却仍然让用户觉得慢；所以技术性能还要引入 latency、duration、错误率等指标。可用性回答“能不能用”，性能回答“用起来是否顺滑”。

## 2.2 Metrics: latency、duration、4xx/5xx error count

Metrics 是可聚合、可比较、可长期观察的指标。传统 observability 最熟悉的指标包括 latency、duration、4xx 错误数和 5xx 错误数。Latency 通常指用户发起请求到看到响应之间的等待时间；duration 更强调一次交互或一个内部步骤从开始到结束的总耗时。二者经常接近，但在复杂 workflow 中，duration 可能包含多个阶段的累积耗时。

4xx 和 5xx 错误提供了另一种分层视角。4xx 通常表示客户端侧问题，例如请求格式错误、未授权、资源不存在；5xx 通常表示服务端侧问题，例如服务异常、依赖失败、内部错误。把两类错误拆开看，团队才能判断问题更像“用户请求不符合要求”，还是“系统处理能力出了故障”。

### Metrics 的价值

Metrics 的核心价值在于聚合。单次请求慢一次只能说明一个样本；当 P95 latency、5xx count 或 error rate 在一段时间里持续升高，团队就能把局部事件提升为系统信号。

可以把传统可观测性在一次请求或工作流上的观察集合压缩为：

$$O_{trad}(\tau) = \{U, L, D, E_{4xx}, E_{5xx}, S_1, \dots, S_n\}$$

- $O_{trad}(\tau)$ ：传统可观测性对一次 trace  $\tau$  收集到的观察集合。
- $\tau$ ：一次请求、一次交互或一次完整工作流的 trace。
- $U$ ：uptime 或 availability，表示系统在该观察周期内是否可用。
- $L$ ：latency，表示用户或上游系统等待响应的的时间。
- $D$ ：duration，表示一次交互或内部步骤从开始到结束的总耗时。
- $E_{4xx}$ ：4xx 客户端错误计数。
- $E_{5xx}$ ：5xx 服务端错误计数。
- $S_i$ ：第  $i$  个 span，表示 trace 中的一个局部步骤。
- $n$ ：该 trace 中 span 的总数。

## 2.3 Trace 与 span：一次 workflow 与其中一步

Trace 是一次完整交互或 workflow 的记录。它像一张物流单：从用户下单开始，到支付、库存、配送、通知等环节结束，整条链路都能串起来。Span 是 trace 里的一个步骤。一次 API 请求可以有数据库查询 span、缓存读取 span、外部服务调用 span、业务计算 span。Trace 负责串联全局故事，span 负责展示局部动作。

这个区分对定位问题非常关键。假设用户觉得一次操作很慢，只看总耗时只能知道“慢了”；trace/span 可以告诉团队慢在数据库、慢在第三方 API、慢在鉴权，还是慢在应用代码本身。传统可观测性成熟的原因之一，就是它把“系统很慢”拆成可定位、可分工、可修复的局部步骤。

### 不要把 trace 当成纯日志

日志通常是一行一行的事件记录，trace 强调因果链路和层级结构。span 之间有父子关系、开始结束时间和上下文传播。没有这些结构，团队只能在日志海里搜索关键词，很难还原一次请求的完整路径。

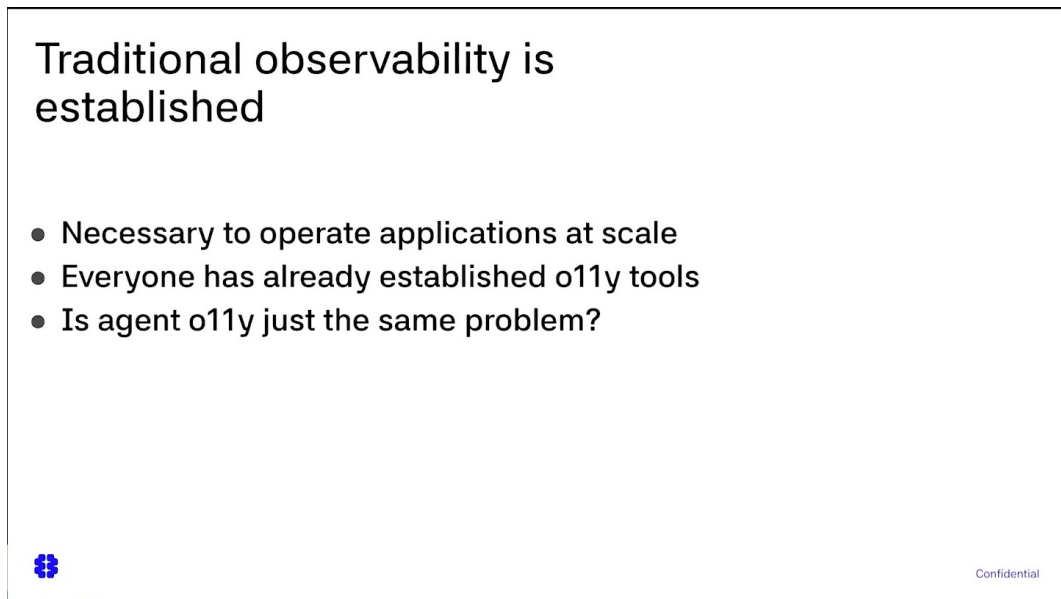


图 3: 传统可观测性已经成熟: 它支持规模化运行, 也让团队自然追问 Agent 可观测性能否沿用同一套工具。<sup>3</sup>

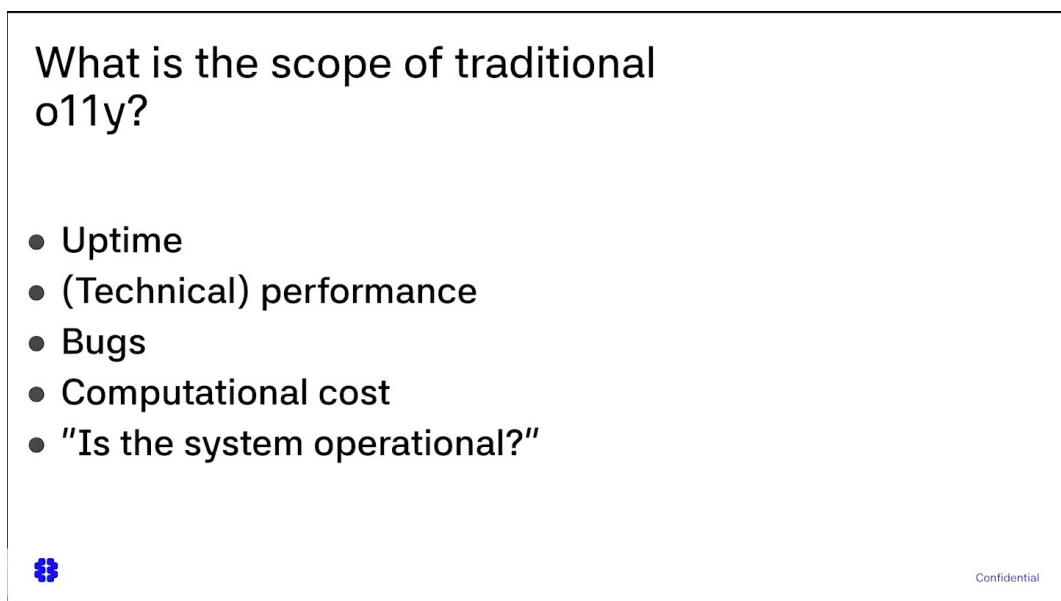


图 4: 传统可观测性的核心范围集中在系统运行状态: 可用性、技术性能、错误和计算成本。<sup>4</sup>

<sup>3</sup>视频画面时间区间: 00:02:43.00–00:03:12.79。

<sup>4</sup>视频画面时间区间: 00:03:16.19–00:03:44.00。

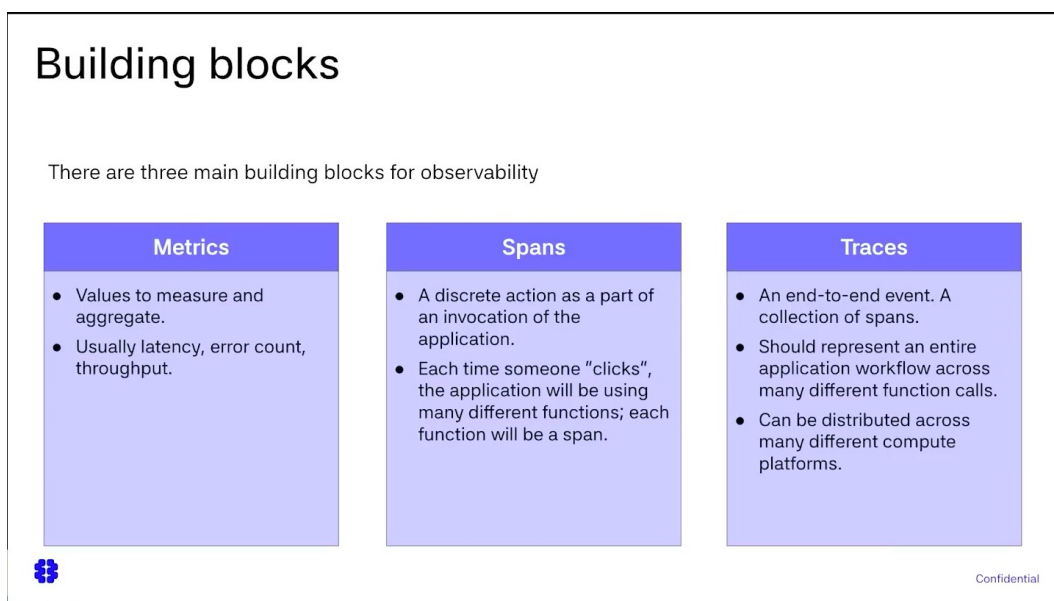


图 5: 可观测性的三个基础构件: 指标用于聚合, Span 描述单步动作, Trace 串起端到端工作流。<sup>5</sup>

## 2.4 Grafana 与 Datadog: 成熟工具适合成熟问题

Hetzel 明确承认传统 observability 的成熟度。他提到 Grafana、Datadog 这样的工具已经非常成熟, Braintrust 自己也使用 Datadog 来观察网站上的 400 或 500 级错误。这一点很重要: 对传统技术运行面, 成熟工具已经提供了有效解法。

Grafana 常被用来展示和组合指标面板, 让团队看到时间序列、阈值告警和趋势变化。Datadog 则提供覆盖基础设施、应用性能、日志、trace 和告警的一体化平台。二者的成熟并不削弱 agent o11y 的必要性, 反而把基线讲清楚: 后续讨论承认传统工具的有效范围, 同时说明 agent 场景把问题空间扩大了。

### 成熟工具的边界意识

一个成熟工具最强的地方, 往往也暴露了它的边界。Grafana 和 Datadog 擅长处理技术指标、错误、延迟、链路追踪; 当团队开始追问“回答是否有依据”“工具调用是否合理”“某类用户意图为什么失败”时, 数据形态和判断标准都发生了变化。

## 2.5 为什么这些 building blocks 仍会被 Agent o11y 继承

Agent o11y 会继续保留 metrics、trace 和 span。相反, 它会继承这些 building blocks。一个 agent 请求同样有延迟、耗时、错误、模型调用、工具调用、缓存命中、外部服务依赖。没有这些基础信号, 团队连 agent 是否正常运行都无法判断。

区别在于继承之后还要扩展。传统 trace 可能记录一次 HTTP 请求和几个服务调用; agent trace 还需要记录 prompt、response、tool call、retrieval context、system instruction、评分结果和人工标注。传统 metrics 关注“快不快、错没错”; agent metrics 还会继续扩展到 time to first token、token 数量、groundedness、tool-use correctness、brand alignment 等质量信号。第 2 节因此承担一个基线作用: 先把传统 observability 的可复用部分讲清楚, 后面的章节才能准确解释 agent 场景新增了什么。

<sup>5</sup>视频画面时间区间: 00:04:29.48–00:04:46.19。

## 传统可观测性提供底座

传统 observability 的底座是 metrics、trace 和 span。Agent o11y 的第一步仍然要站在这个底座上，随后把观察对象从技术运行扩展到语义行为、工具使用、质量评估和迭代反馈。

## 2.6 本章小结

本章定义了传统 observability 的边界：它主要回答系统是否在线、技术性能是否达标、错误是否升高、一次 workflow 中哪个 span 出了问题。Grafana、Datadog 代表了这一领域的成熟工具，适合 uptime、latency、duration、4xx/5xx error count、metrics、trace/span 等典型问题。这个底座会被 Agent o11y 继承；后续真正要展开的是，agent 应用在语义质量、非确定性行为、长文本 trace 和多角色协作上提出了额外要求。

## 3 第一差异：Agent 的非确定性与质量指标扩张

### 3.1 同一套 building blocks，换了一类行为对象

Phil Hetzel 在这里先做了一个关键转折：metrics、traces、spans 这些 building blocks 仍然适用于 agent observability。换句话说，agent 系统并没有抛开传统可观测性的底座。一次完整交互仍然可以记为一个 trace，模型调用、工具调用、检索、后处理等步骤仍然可以拆成 span，延迟、持续时间、错误计数仍然可以被聚合成指标。

真正改变的是被观测对象的行为形态。传统应用通常围绕确定性的控制流运行：路由、条件分支、数据库查询、异常处理都写在代码里。给定同一个输入、同一份数据库状态和同一组配置，工程师大体能预测程序会走哪条路径。它像一条有岔道的铁路，岔道很多，但每个岔道由显式信号灯控制。

Agent 应用的路径空间更像一个带导航系统的城市行程。目标相同，路线可能有多条：模型可以先检索再回答，也可以先澄清问题；可以调用搜索工具，也可以调用知识库；可以用不同措辞给出同样正确的答案。这种非确定性来自 LLM 的高变化能力和抽象推理能力。这里的“非确定性”指多条有效路径、多种合格输出和不同中间推理轨迹，仍然需要规则、评测和治理。

## 重点提示

Agent observability 的第一层难点在于：工程团队既要问“程序是否按固定路径执行”，也要问“在多个可能路径中，agent 为什么选择了这一条，结果是否仍然正确、可靠、合规”。这把可观测性从技术运行面推进到行为质量面。

### 3.2 确定性控制流与 LLM 高变化行为

“确定性应用”里的 deterministic control flow，核心含义是控制权主要掌握在开发者写下的程序结构中。一个电商结算接口收到请求后，通常会依次校验库存、计算价格、创建订单、扣减库存、发起支付。若某一步失败，代码会进入事先定义的错误分支。传统 observability 关注这些节点是否变慢、失败、重试或超时。

LLM agent 的 high variety behavior 则把一部分控制权交给模型选择。模型根据 prompt、上下文、工具描述、历史消息和中间结果决定下一步。这个选择可以有价值，因为它让 agent 处理模糊需求、开放问题和未预设任务；这个选择也带来观测压力，因为工程师需要理解“为什么这次没有调用工具”“为什么用了这个检索结果”“为什么回答风格偏离了系统提示”。

## 补充知识

可以把传统应用理解为“菜谱式流程”：每一步顺序清楚，偏差通常意味着 bug、配置问题或依赖故障。Agent 更像“专业助理式流程”：目标明确，但助理会根据现场材料决定先查资料、先问问题、先算账，还是直接起草答案。评估助理需要同时看动作快慢和判断依据。

这种差异可以压缩成一个观测集合的扩张：

$$O_{agent}(\tau) = O_{trad}(\tau) \cup \{TTFT, N_{tok}, G, T_{use}, B, Q\}$$

- $\tau$ ：一次完整的 agent trace，也就是从用户输入到最终输出的交互记录。
- $O_{agent}(\tau)$ ：针对这次 agent trace 需要收集和判断的观测信号集合。
- $O_{trad}(\tau)$ ：传统可观测性中的观测信号集合，例如 latency、duration、error count、span timing。
- $\cup$ ：集合并集，表示 agent observability 继承传统指标，并额外加入新的质量指标。
- $TTFT$ ：time to first token，用户发出请求后到模型吐出第一个 token 的时间。
- $N_{tok}$ ：total tokens，本次交互中输入、输出和中间调用消耗的 token 数量。
- $G$ ：groundedness，回答是否扎根于上下文证据、检索内容或工具返回结果。
- $T_{use}$ ：tool-use correctness，工具选择、参数传入、结果读取和后续使用是否正确。
- $B$ ：brand alignment，输出语气、用词边界、拒答风格和品牌标准是否一致。
- $Q$ ：业务定义下的整体 agent quality score，可能由人工标注、自动评分函数或两者组合得到。

### 3.3 从“快不快”扩展到“答得好不好”

演讲中先列出的 TTFT、total tokens、duration、latency，听起来很像传统 observability 的指标。它们仍然重要。TTFT 影响用户是否感觉系统“有反应”；total tokens 影响成本、上下文窗口占用和吞吐；duration 与 latency 影响交互节奏和服务容量。

但这些指标只能回答“系统运行状态如何”，无法回答“agent 这次表现是否合格”。一个 agent 可以在 500 毫秒内给出非常流畅的错误答案，也可以低成本地省略关键工具调用，还可以稳定地输出不符合品牌风格的措辞。传统应用的 4xx/5xx 错误码在这里不够用，因为很多失败会以“语义上看似顺畅”的形式出现。

## Agent applications are non-deterministic

- Typical applications have deterministic code paths.
- There could be control flow but given some input, you will always traverse the same path.
- Related - unit testing these applications is straightforward and also deterministic

### Traditional observability is meant to measure deterministic metrics and codepaths

Confidential

图 6: Agent 应用具有非确定性，同一任务附近的上下文和模型采样可能触发不同代码路径与工具调用。<sup>6</sup>

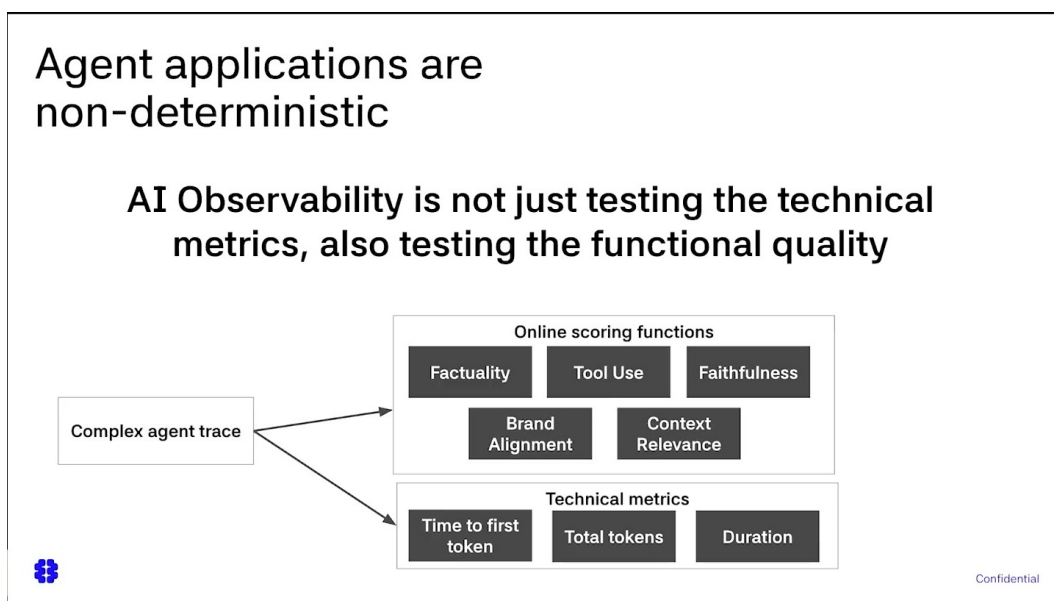


图 7: Agent 可观测性除了延迟、token 数、耗时等技术指标，还要评估事实性、工具使用、忠实度、品牌一致性与上下文相关性。<sup>7</sup>

#### 易错提醒

常见误读是把 agent 的质量问题都折算成延迟、错误率或 token 成本。这样会漏掉更危险的失败：回答无证据、工具调用顺序错误、引用了过期上下文、语气违反品牌要求。这些失败不会必然触发 HTTP 500，却会直接伤害用户信任。

### 3.4 Groundedness: 回答是否扎根于证据

Groundedness 衡量回答中的关键主张是否能在上下文中找到支撑。这里的上下文可以是检索到的文档、数据库查询结果、工具返回值、用户先前消息或系统预置知识。它关心的是“答案有没有根据”，像法庭上的证据链：结论可以有表达差异，但关键事实需要能追溯到材料。

一个简化评分可以写成：

$$G(\tau) = \frac{|C_{supported}(\tau)|}{|C_{all}(\tau)|}$$

- $G(\tau)$ : trace  $\tau$  的 groundedness 分数。
- $\tau$ : 一次完整 agent 交互的 trace。
- $C_{supported}(\tau)$ : 回答中能被上下文证据支持的关键主张集合。
- $C_{all}(\tau)$ : 回答中所有需要事实支撑的关键主张集合。
- $|\cdot|$ : 集合大小，也就是集合中元素的数量。

这个公式只是教学抽象。真实系统里，主张抽取、证据匹配、引用强度、上下文冲突都会让 groundedness 更复杂。它的价值在于提醒读者：agent observability 需要保存足够多的 prompt、retrieved context、tool result 和 final response，否则系统连“答案是否扎根”都无从判断。

<sup>6</sup>视频画面时间区间：00:04:46.19–00:05:29.83。

<sup>7</sup>视频画面时间区间：00:06:16.19–00:06:56.00。

### 3.5 Tool-use correctness: 工具调用是否合乎任务

Tool-use correctness 衡量 agent 是否在合适的时机调用了合适的工具，并且正确使用工具结果。这里有四个层次：

- 工具选择：任务需要查库存时是否调用库存工具，需要算税时是否调用税务计算工具。
- 参数构造：调用工具时是否传入正确用户、时间范围、筛选条件和业务实体。
- 结果解释：工具返回空结果、错误结果或多候选结果时，agent 是否正确处理。
- 后续使用：最终答案是否忠实使用工具结果，没有编造额外字段或忽略关键警告。

传统 observability 可以告诉工程师某个 API 调用耗时多少、是否超时、返回码是多少。Agent observability 还要告诉团队：这次工具调用在语义上是否应该发生，发生后是否被正确理解。这里的 span 需要带上工具名、输入参数、输出文本、模型随后的解释和最终答案，否则只能看到“调用成功”，看不到“用得是否正确”。

### 3.6 Brand alignment: 系统提示与品牌边界是否被执行

Brand alignment 指 agent 输出是否符合组织设定的语气、边界和行为规范。它同时涉及营销口吻和风险控制。例如客服 agent 是否承诺了公司无法兑现的政策，医疗 agent 是否使用了过度确定的诊断措辞，金融 agent 是否越过了投资建议边界。

这一指标和系统提示密切相关。系统提示像服务人员的岗位手册，规定语气、禁区、升级路径和必须披露的信息。Agent 可能完成了任务，也可能回答了事实问题，但如果输出风格、承诺范围或免责声明违背这份手册，质量仍然不合格。

#### 补充知识

读者容易忽略一个“未知的未知”：品牌一致性常常超出单句规则。它可能藏在行业合规、客户分层、产品定位、法律免责声明和公司语气指南之间。一个好用的 agent observability 平台需要让这些软约束变成可检查、可回放、可讨论的信号。

### 3.7 本章小结

本章解释了第一个差异：agent 的非确定性来自 LLM 的高变化能力和多路径推理空间，它描述的是“多种合理行为”的存在，仍然需要规则、评测和治理。传统应用的控制流通常由代码显式规定，传统 observability 因此擅长测量 uptime、latency、duration 和 error count。Agent observability 需要继续保留这些指标，同时扩展到 TTFT、total tokens、groundedness、tool-use correctness、brand alignment 和业务质量分数。判断一个 agent 是否可靠，必须同时看技术表现和语义质量。

## 4 第二差异：Agent trace 的形状、体积与实时压力

### 4.1 质量指标扩张会反向改变 trace 的数据形状

第 4 章接续上一章的指标扩张。若团队只关心 latency 和 error count，trace 可以比较瘦：请求什么时候进来，哪个服务处理，哪个 span 耗时多少，返回码是多少。若团队要判断 groundedness、工具使用正确性和品牌一致性，trace 就必须保存更多上下文：用户原始问题、系统提示、模型输入、模型输出、检索证据、工具参数、工具返回文本、模型中间步骤和最终回复。

这就是 Phil Hetzel 所说的 agent traces “nasty” 的第一层含义：它们高度半结构化。半结构化数据指数据有一定字段和层级，例如 trace id、span id、timestamp、model name、tool name、status、duration；同时字段内部又塞入大量自然语言文本，例如 prompt、response、tool output

和 retrieved context。这类数据像一份案件卷宗：封面、目录、时间线是结构化的，证词、证据摘录、专家意见和判定理由是长文本。

### 重点提示

Agent trace 的难点包括“字段更多”，也包括结构化元数据和非结构化长文本混在同一个交互记录里：系统既要按时间线重放，也要按字段过滤，还要对 prompt、response 和 tool output 做语义检查。

## 4.2 半结构化外壳与非结构化文本内核

可以把一个 agent trace 抽象成下面的组合：

$$\tau = \{M, S, P, R, T, E\}$$

- $\tau$ ：一次完整的 agent trace。
- $M$ ：结构化元数据，例如 trace id、用户会话、时间戳、环境、模型名称。
- $S$ ：span 序列，例如模型调用、工具调用、检索步骤、后处理步骤。
- $P$ ：prompt 文本，包括 system prompt、developer prompt、user message 和中间提示。
- $R$ ：response 文本，包括模型中间输出和最终回复。
- $T$ ：tool data，包括工具名称、参数、返回值、错误信息和重试记录。
- $E$ ：evidence，上下文证据集合，例如检索片段、数据库结果和外部 API 返回内容。

传统 trace 更像收据：每一行是服务名、耗时、状态码。Agent trace 更像卷宗：同样有时间线，但真正决定质量的内容埋在文本、工具返回和证据链里。若系统只保存 span 名称和耗时，就会丢失“为什么 agent 给出这个答案”的关键材料。

## 4.3 从 KB 级记录到 GB 级 trace、20MB span

演讲给了两个很硬的数字：一个 agent trace 可能超过 1GB，单个 span 可能达到 20MB。这两个数字的意义不在于制造夸张感，而在于说明系统设计量级发生了变化。一个 20MB span 可能包含长 prompt、大段工具返回、检索出来的文档片段、模型输出和调试信息。一个超过 1GB 的 trace 可能来自长会话、多轮工具调用、大量检索证据、批量处理或复杂 agent workflow。

体积增长可以用一个粗略分解理解：

$$Size(\tau) = Size(M) + \sum_{i=1}^n Size(S_i) + Size(P) + Size(R) + Size(T) + Size(E)$$

- $Size(\tau)$ ：完整 trace  $\tau$  的数据体积。
- $M$ ：trace 的结构化元数据。
- $n$ ：trace 中 span 的数量。
- $S_i$ ：第  $i$  个 span。
- $Size(S_i)$ ：第  $i$  个 span 的数据体积。
- $P$ ：prompt 文本集合。
- $R$ ：response 文本集合。
- $T$ ：工具调用相关数据集合。
- $E$ ：上下文证据集合。
- $\sum_{i=1}^n$ ：把第 1 个到第  $n$  个 span 的体积相加。

这个分解揭示了一个容易被低估的问题：agent trace 的大体积常常由多个来源叠加形成。压缩某一个字段只能缓解一部分压力。从工程实现看，系统还要考虑采样、截断、分层存储、索引策

略、权限控制和 UI 渲染；这属于基于演讲中体积与实时性压力做出的工程外推。否则 trace 存进去了，工程师和产品经理也可能打不开、搜不到、看不懂。

#### 易错提醒

不要把“1GB trace”理解成单纯的存储账单问题。它同时影响写入吞吐、查询延迟、索引构建、浏览器渲染、权限审计和在线评分。体积越大，越需要明确哪些文本必须保留，哪些内容可以摘要，哪些字段必须索引。

#### 4.4 实时压力：ingest、process、use 必须同时成立

Phil Hetzel 特别强调，agent trace 虽然更大、更乱，但速度要求没有降低。生产中的 agent 一旦获得 product-market fit，就会产生大量用户交互。AI engineer 或产品经理会希望在 true real time 里看到这些交互：用户刚完成一次对话，团队就能在 UI 中看到 trace，检查模型调用、工具调用和输出质量。

这带来三个并发目标：

- ingest: trace 要快速进入系统，不能在采集入口长期排队。
- process: 系统要解析 span、抽取字段、处理长文本、准备索引和评分材料。
- use: 用户要能立刻在 UI 或查询接口中查看、过滤、搜索和分析。

如果只满足 ingest，数据虽然进来了，用户仍然看不到有效视图。如果只满足 process，后台很完整，前台延迟会让排障错过时机。如果只满足 use，系统可能牺牲数据完整性，后续 groundedness 和 tool-use correctness 的判断就失去材料。

实时约束可以表达为：

$$L_{visible} = L_{ingest} + L_{process} + L_{read} \leq L_{SLO}$$

- $L_{visible}$ : 一条 trace 从产生到用户可见的总延迟。
- $L_{ingest}$ : 采集和写入入口的延迟。
- $L_{process}$ : 解析、转换、索引准备和评分准备的处理延迟。
- $L_{read}$ : UI 或查询接口读取并展示 trace 的延迟。
- $L_{SLO}$ : 团队承诺或期望的可见性服务目标。
- $\leq$ : 小于或等于，表示总延迟必须落在服务目标以内。

#### 4.5 Braintrust trace UI: model call、tool call 与长文本同屏出现

演讲展示的 Braintrust trace UI 用来说明这种数据形状。界面里同时包含许多 span、model calls 和 tool calls；进入 span 后，还能看到大量非结构化文本。这个画面很关键，因为它把抽象问题落到了操作界面：观测者需要沿着时间线看 agent 做了什么，又要钻进每个步骤看模型读了什么、工具回了什么、最终答案如何形成。

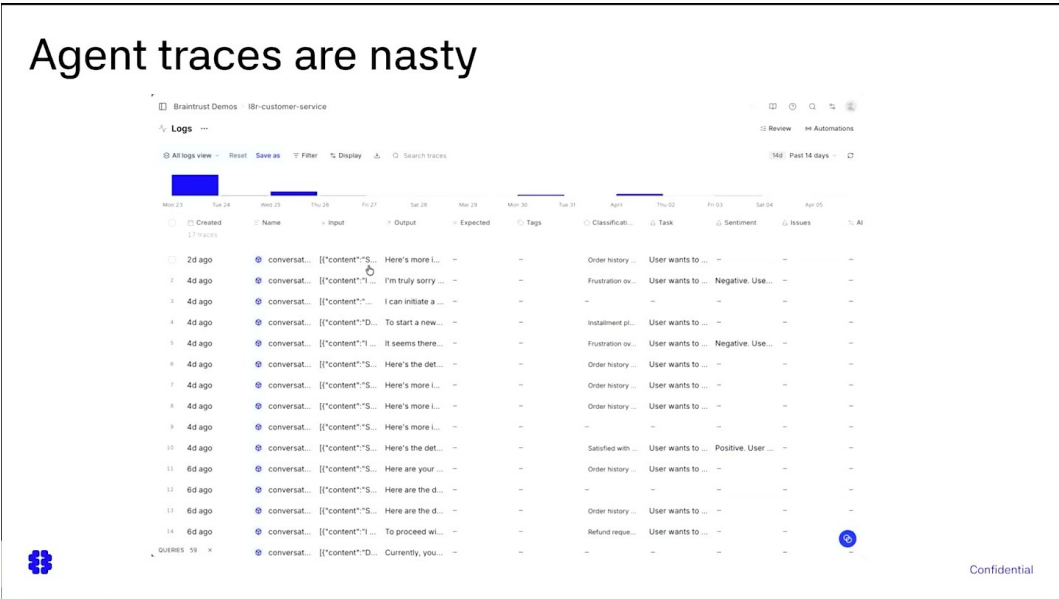


图 8: Braintrust 的 trace 列表把输入输出、分类、任务、情绪与问题聚合到同一张表中，体现 Agent trace 的半结构化特征。<sup>8</sup>

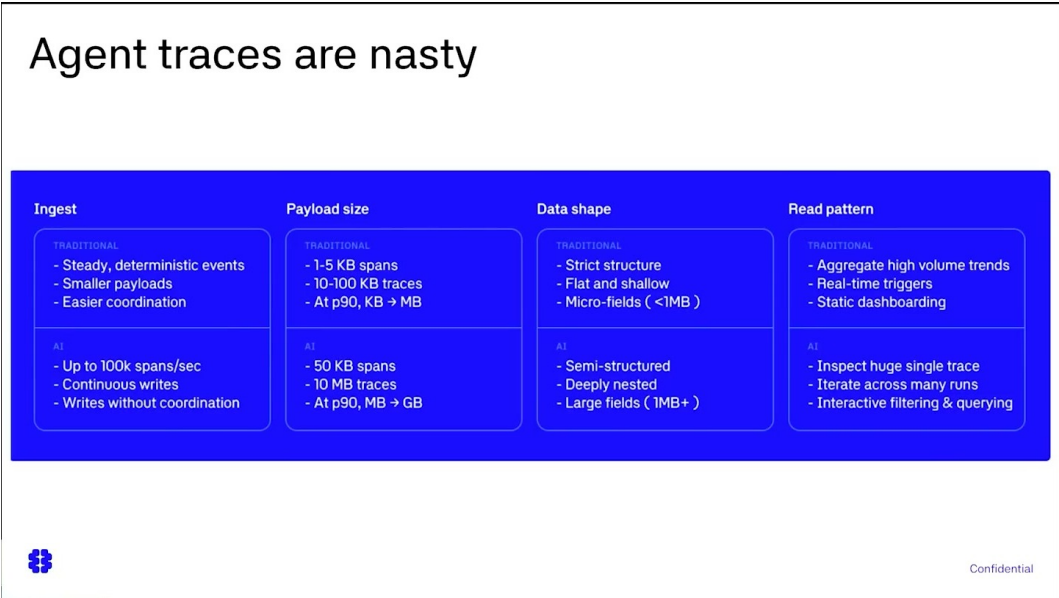


图 9: Agent trace 的系统挑战集中在四个维度：高频写入、更大的 payload、半结构化深层嵌套，以及交互式过滤和查询。<sup>9</sup>

<sup>8</sup>视频画面时间区间：00:08:30.00–00:08:49.83。

<sup>9</sup>视频画面时间区间：00:07:46.19–00:09:16.19。

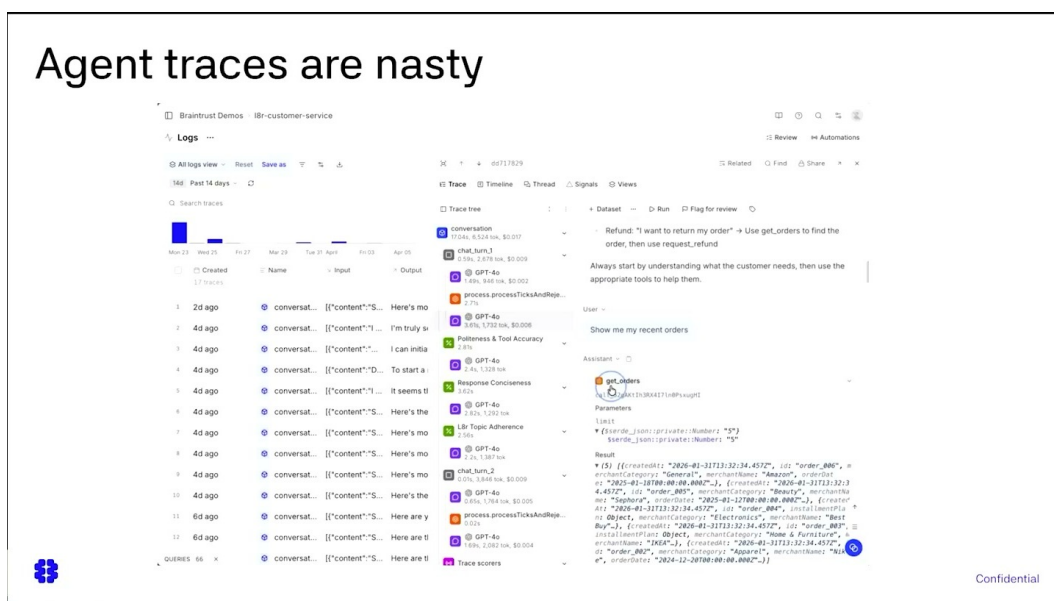


图 10: Braintrust trace 详情页展示 span 树、工具调用参数和返回结果，说明 Agent 可观测性必须处理模型、工具与文本结果的组合。<sup>10</sup>

这种 UI 的价值包括日志展示、质量复盘和系统排障三个工作：

- 时间线重放：让工程师看到 agent 的执行顺序，定位哪一步开始偏离预期。
- 语义审查：让产品经理或领域专家阅读 prompt、response、tool output，判断质量问题。
- 系统排障：让平台团队识别慢 span、大 span、错误工具调用和异常数据形状。

由此可以理解为什么传统 observability 工具在这里会吃力。它们擅长聚合指标、服务拓扑和错误率，但 agent trace 需要把调用链、长文本、工具证据和质量判定放在同一个分析表面上。缺少这个表面，团队只能看到“模型调用成功”，看不到“模型成功地做错了什么”。

## 4.6 系统挑战图中的量级对比

图中的数字还补了一层口播没有逐项念出的系统量级对比。这里必须写成 100k spans/sec，含义是每秒最多约十万条 span 的写入压力；它描述的是 span 写入吞吐量，不能理解为请求吞吐量。在传统 observability 里，span 常见量级约为 1–5KB，trace 常见量级约为 10–100KB；即使到 p90 分位，通常也只是从 KB 进入 MB。AI/agent trace 的量级更激进：图中给出约 50KB 的 span、约 10MB 的 trace，并且 p90 可能从 MB 进入 GB。

这组数字说明，agent trace 的挑战同时来自写入频率和单条记录体积。若把 trace 想成包裹，传统监控处理的是大量小包裹，尺寸稳定、标签整齐；agent observability 处理的是大量厚案卷，单件更重，里面还夹着长文本、工具返回、模型输出和评分材料。物流系统一旦从小包裹变成厚案卷，分拣、索引、存放和检索都会变成新问题。

图中也给出了数据形状和读模式差异。传统数据通常更严格、扁平、浅层，单字段往往小于 1MB；agent 数据更半结构化、深层嵌套，单字段可能超过 1MB。传统读模式偏向聚合高流量趋势、实时触发器和静态 dashboard；agent observability 需要检查巨大的单条 trace，跨多次运行迭代，并支持交互式过滤与查询。

## 4.7 本章小结

本章解释了第二个差异：agent trace 的形状、体积和实时压力共同改变了系统问题。Agent trace 具有半结构化外壳和非结构化文本内核，既要保存 trace id、span、timestamp、model call、

<sup>10</sup>视频画面时间区间：00:10:46.19–00:11:15.00。

tool call 等结构化字段，也要保存 prompt、response、tool output、retrieved context 等长文本。演讲中提到的 1GB trace 和 20MB span 说明这已经超出普通轻量日志的量级。更关键的是，用户仍然要求 true real time: trace 要被快速摄入、处理、索引、读取和展示。Agent observability 因而需要同时解决 ingest、process、use 三件事。

## 5 数据系统方案：WAL、索引、Tantivy 与 SQL 查询

前一章已经说明了 Agent trace 的形状：它既有结构化字段，也有长 prompt、长 response、tool output 这类自然语言文本；它既要承载质量判断，也要在生产环境中近实时出现。Phil Hetzel 在 00:09:16–00:12:16 这一段把问题推进到数据系统层面：当 trace 变成这种“厚案卷”时，传统监控系统里的简单时间序列读写模型就不够用了，系统必须同时服务两类读模式。

第一类读模式面向界面。用户刚刚触发了一次 agent 交互，工程师、产品经理或者值班人员希望在 UI 中几乎立刻看到这条 trace。这里的关键词是“立即可见”：它像急诊分诊台，病人刚到，系统至少要先登记、编号、能查到，后续的详细化验可以继续跑。

第二类读模式面向自动化分析。有人会通过 CLI 发起 SQL 或类 SQL 查询，把 observability trace 或 eval trace 拉进自己的改进流水线。这里的读者未必是浏览器里的一个人，也可能是一段定时任务、一个实验脚本、一个自动化回归评测流程。系统面对的是一组同时存在的访问介质，范围已经超过单一仪表盘。

### 本章的核心判断

Agent observability 的数据层必须同时满足三件事：新 trace 要快速可见，过滤与分析查询要低延迟，大量自然语言文本要被全文检索。WAL、分析索引、Tantivy 全文索引和 SQL-like 查询层分别对应这三类压力。

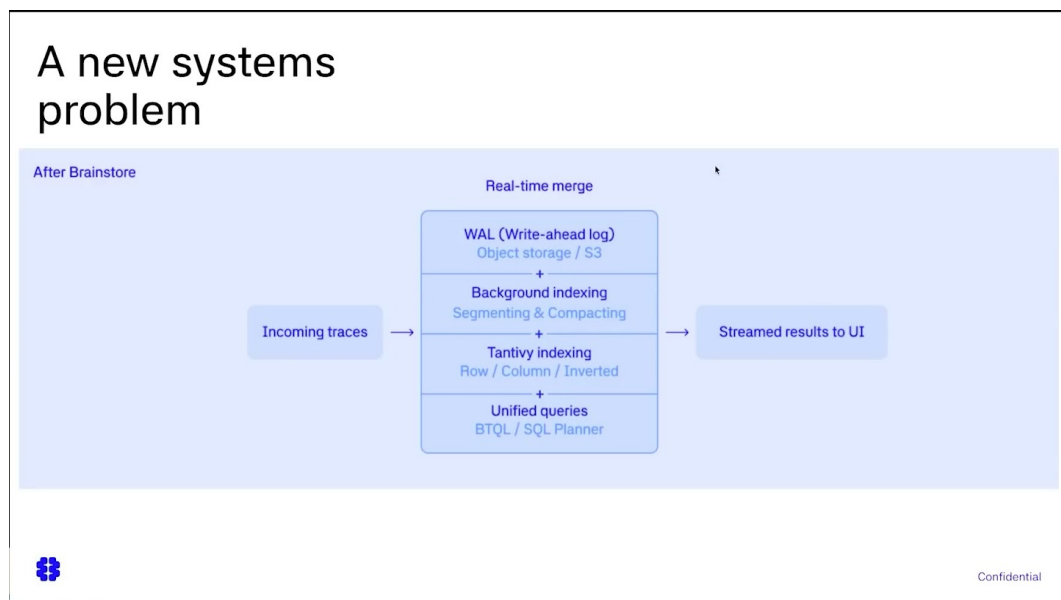


图 11: Braintrust 的 Agent trace 数据库链路：trace 进入 WAL，再经过后台索引、Tantivy 全文索引和统一查询层，最终流式返回 UI。<sup>11</sup>

<sup>11</sup> 视频画面时间区间：00:09:16.19–00:10:35.00。

## 5.1 两种读模式：实时 UI 与 CLI/SQL 分析

在传统服务监控里，常见读模式是“看图”：打开面板，观察延迟、错误率、吞吐量曲线是否异常。Agent observability 保留这种面板需求，同时多出一种更像数据仓库和搜索引擎结合体的需求：用户希望对 trace 本身进行查询、过滤、分组、抽样和回放。

可以把读模式压缩成一个简单集合：

$$Read(\tau) = \{R_{ui}(\tau), R_{cli}(\tau), R_{sql}(\tau), R_{search}(\tau)\}$$

- $\tau$ ：一条 agent trace，表示一次完整交互或工作流记录。
- $Read(\tau)$ ：围绕这条 trace 产生的读需求集合。
- $R_{ui}(\tau)$ ：面向 UI 的实时查看需求，例如在浏览器里立刻打开最新交互。
- $R_{cli}(\tau)$ ：面向命令行的读取需求，例如脚本拉取 trace 后进入自动分析流程。
- $R_{sql}(\tau)$ ：面向 SQL 或类 SQL 的结构化查询需求，例如按模型、时间、标签、评分过滤。
- $R_{search}(\tau)$ ：面向自然语言文本的搜索需求，例如查找包含某个词的所有 trace。

这个公式表达的是“同一份 trace 被不同入口读取”。如果系统只为 UI 做优化，自动化分析会被卡住；如果系统只为离线分析做优化，用户又会觉得生产现场看不见、跟不上。Braintrust 在这一段强调“不同媒介会查询这种巨大的 trace 形状”，含义就在这里。

## 5.2 Write-ahead log：先让 trace 进入可见轨道

Write-ahead log，简称 WAL，中文常译为预写日志。它的直觉很朴素：先把即将发生的数据变更写进一条顺序日志，再做更复杂的整理、索引和持久化布局。它像仓库收货时先扫条码打入流水账，货架归位、分类标签、盘点报表可以随后完成。这样做的价值是降低“刚发生的事情查不到”的风险。

在这段演讲里，WAL 的功能重点放在 agent trace 的实时可见性上，不展开数据库教材里的全部事务理论：当用户期望“我刚刚执行的 agent 动作马上出现”时，系统先把 trace 放进可读的日志轨道，后续索引可以异步推进。

$$t_{visible} \approx t_{ingest} + t_{wal}$$

- $t_{visible}$ ：trace 对用户或查询系统可见的时间点。
- $t_{ingest}$ ：系统开始接收 trace 的时间点。
- $t_{wal}$ ：把 trace 写入预写日志并完成最小可读登记所需的时间。
- $\approx$ ：这里表示近似关系，用来强调可见性主要受接收与 WAL 写入路径影响。

这个公式刻意忽略后续全文索引、聚合索引、压缩和长期存储，因为 UI 的第一诉求是“先看到”。如果把所有加工都放在可见性之前，系统就会像要求病历全部归档完成后才允许医生看患者信息，流程看似整齐，响应会很慢。

### 不要把 WAL 理解成完整答案

WAL 解决的是快速登记和恢复边界，它不会自动让过滤、聚合和全文搜索变快。Agent trace 的难点在于“先看见”之后还要“查得动”，所以 WAL 后面必须接索引体系。

## 5.3 过滤与分析索引：给结构化字段修目录

当用户按时间、模型、环境、项目、评分、错误状态、工具名称过滤 trace 时，系统查询的是结构化或半结构化字段。没有索引时，系统只能像翻一整柜案卷一样逐条检查；有索引后，它可以先查目录，直接定位候选集合。

过滤索引服务的问题可以写成：

$$F(p) = \{\tau_i \mid p(\text{meta}(\tau_i)) = \text{true}\}$$

- $F(p)$ : 满足过滤条件  $p$  的 trace 集合。
- $p$ : 过滤谓词, 例如“模型等于某版本”“评分低于阈值”“时间位于某个窗口”。
- $\tau_i$ : 第  $i$  条 agent trace。
- $\text{meta}(\tau_i)$ : trace 的结构化元数据, 例如时间戳、标签、模型名、项目名、状态码、评分字段。
- $\text{true}$ : 谓词判断为真, 表示该 trace 命中过滤条件。

分析索引的作用还会延伸到聚合, 例如统计某版本 agent 的失败率、比较不同 prompt 的平均评分、筛出某个工具调用失败后的回答质量。它和全文索引的分工不同: 分析索引更像表格里的列索引, 全文索引更像一本书末尾的词条索引。

#### 过滤索引与全文索引的区别

过滤索引关注字段值, 例如时间、项目、状态、分数; 全文索引关注词项出现在哪里, 例如 prompt 或 tool output 里是否出现了 Amazon。前者像数据库表的列目录, 后者像搜索引擎的倒排表。Agent trace 同时需要两者, 因为它一半像表格, 一半像长文档。

### 5.4 Tantivy 全文索引: 为什么“Amazon in trace”是中心例子

Phil 在这一段给出一个很具体的例子: 用户想知道“哪些 trace 里出现过 Amazon 这个词”。这个例子看似简单, 实际暴露了 Agent observability 数据系统的关键压力。

如果 trace 只包含延迟、错误码、请求路径, 搜索 Amazon 可能没有太大意义。Agent trace 里却可能有用户输入、检索结果、网页摘要、工具返回、模型推理上下文、最终回答。Amazon 可能出现在用户问题里, 可能出现在检索证据里, 也可能出现在 agent 调用某个工具后的返回文本里。想找出它, 系统需要跨越大量长文本字段。

全文检索通常依赖倒排索引。倒排索引的直觉是: 先把“词  $\rightarrow$  文档列表”建好。查询 Amazon 时, 系统不用逐条扫完整 trace, 只要查 Amazon 这个词项对应的 trace 列表。

$$R(q) = \{\tau_i \mid q \in \text{Text}(\tau_i)\}$$

- $R(q)$ : 命中文本查询词  $q$  的 trace 集合。
  - $q$ : 文本查询词, 例如 Amazon。
  - $\tau_i$ : 第  $i$  条 agent trace。
  - $\text{Text}(\tau_i)$ : trace 内所有可检索文本的集合, 包括 prompt、response、tool output、span 字段中的长文本。
  - $\in$ : 属于关系, 表示查询词出现在某条 trace 的可检索文本中。
- 进一步看, 倒排索引可以写成:

$$\text{Inv}(q) = \{\text{id}(\tau_i) \mid q \in \text{Text}(\tau_i)\}$$

- $\text{Inv}(q)$ : 查询词  $q$  对应的倒排列表。
- $q$ : 被索引的词项, 例如 Amazon。
- $\text{id}(\tau_i)$ : 第  $i$  条 trace 的唯一标识符。
- $\tau_i$ : 第  $i$  条 agent trace。
- $\text{Text}(\tau_i)$ : 从 trace 中抽取出来参与全文索引的文本集合。
- $|$ : 集合条件分隔符, 表示右侧条件成立的元素会进入集合。

Tantivy 就处在这个位置。演讲中 Phil 解释它是一个用于 text-style indexing 的开源框架, Braintrust 对它做了 fork; 现场有人追问它和开源搜索技术的关系, Phil 补充说它最像 Apache

Lucene，只是用 Rust 写成。这个类比很有用：Lucene 可以被理解为许多搜索系统背后的底层检索库，Tantivy 则是 Rust 生态里的类似构件。

#### Amazon 搜索例子的系统含义

“找到所有包含 Amazon 的 trace” 远超一个普通字符串匹配小功能。它要求系统把 agent 交互中的长文本拆词、建索引、维护 trace 标识、支持快速召回，并把结果和结构化过滤条件一起返回。这个例子把 Agent observability 从监控面板推向搜索系统和分析系统的交叉地带。

#### 关于 ClickHouse 或 OLAP 的边界

本章只根据这段演讲讨论 Braintrust 面对 agent trace 文本检索压力时选择的系统设计；这里不泛化评价某类 OLAP 系统在所有 Agent observability 场景中的适配性。这里的关键点是：在该场景下，全文索引是一等需求，不能被当作附属功能处理。

### 5.5 SQL-like 统一查询层：把多种能力收束成一种入口

WAL 负责快速可见，分析索引负责字段过滤和聚合，Tantivy 负责全文检索。系统还需要一个统一入口，让这些能力能被同一套查询语言组织起来。Braintrust 在演讲中说他们选择了 SQL 或 SQL-similar language 这条路线；这里描述的是查询层抽象，不能直接等同于产品内部实现细节。

这个选择的工程价值很直接。SQL-like 接口把不同读模式接到同一个抽象层上：UI 可以生成查询，CLI 可以提交查询，自动化脚本也可以提交查询。用户不必分别学习“实时日志读取语言”“字段过滤语言”“全文检索语言”。理想状态下，他只需要表达：时间范围是什么，过滤条件是什么，文本中要找什么，返回哪些字段，如何排序或聚合。

可以把统一查询看成一次组合：

$$Q = \text{Select}(\text{Project}(\text{Filter}(\text{Search}(\text{WAL} \cup \text{Store}))))$$

- $Q$ ：一次完整查询。
- $WAL$ ：预写日志中的近期 trace 数据。
- $Store$ ：已经进入长期存储或主存储布局的 trace 数据。
- $\cup$ ：并集，表示查询可能同时覆盖近期数据和已整理数据。
- $\text{Search}(\cdot)$ ：全文检索步骤，例如搜索 Amazon。
- $\text{Filter}(\cdot)$ ：结构化过滤步骤，例如限制项目、时间范围、模型版本或评分。
- $\text{Project}(\cdot)$ ：字段选择步骤，例如只返回 trace id、时间、评分、命中文本片段。
- $\text{Select}(\cdot)$ ：对外呈现结果的查询操作抽象。

这个公式作为教学抽象使用，用于表达统一查询层的职责：把日志、存储、字段索引、全文索引和返回格式连成一条用户可用的路径。对使用者来说，SQL-like 接口像一个总问诊台；对系统来说，它背后连接的是不同性质的索引和存储部件。

### 5.6 本章小结

第 5 章说明了 Agent observability 为什么会变成数据系统设计问题。演讲中的关键链条是：trace 体积大、文本多、读模式多，所以系统要同时支持实时 UI 可见性、CLI/SQL 分析读取、结构化字段索引和全文搜索。

WAL 让新 trace 尽快进入可见轨道；分析索引让过滤、聚合和低延迟分析可行；Tantivy/Lucene 类全文索引让“找出所有包含 Amazon 的 trace”这种自然语言搜索变成可规模化操作；SQL-like

查询层把这些能力压成统一入口。这里的数据库设计被 trace 的形状牵引，也被使用者的查询方式牵引。

## 6 第三个差异：使用者从工程师扩展到领域专家

00:12:16–00:13:46 这一段切到第三个差异：Agent observability 的使用者群体发生了变化。传统 observability 的典型使用者是系统工程师、平台工程师、产品工程师，核心问题围绕 uptime、latency、error rate、资源消耗和技术体验。Agent observability 如果做得好，使用者会扩展到 subject matter experts，简称 SME，即领域专家。

领域专家这个词很具体。医疗场景里，它可能是 clinician 或 registered nurse；财富管理场景里，它可能是 wealth advisor；法律场景里，它可能是 lawyer；产品场景里，它可能是 product manager 或一线运营人员。这些人不一定会写后端服务，不一定会调 OpenTelemetry pipeline，但他们知道什么回答会伤害用户、什么建议违反业务规范、什么措辞会造成法律或医疗风险。

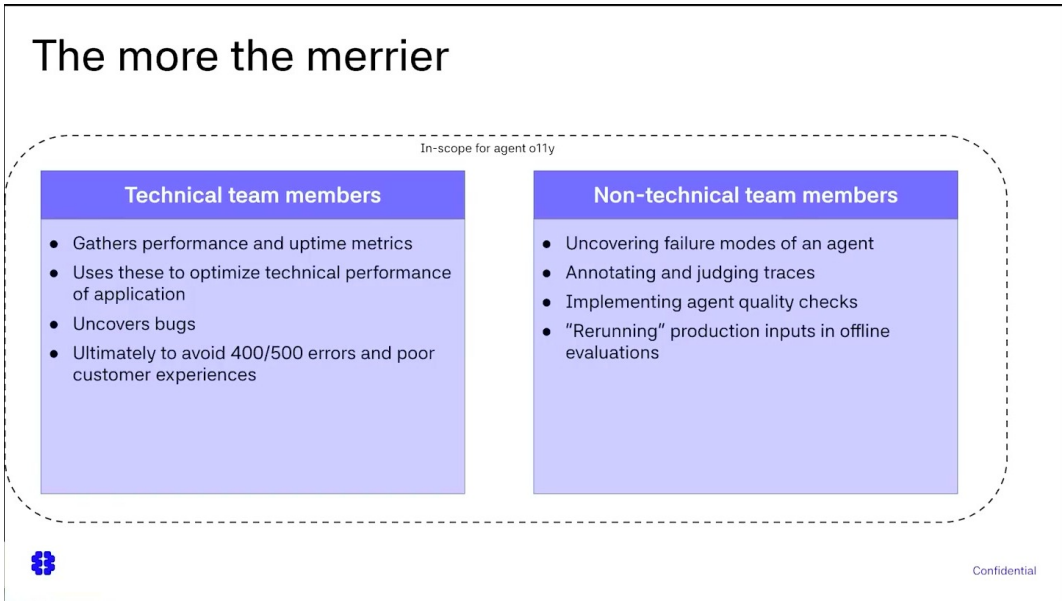


图 12: Agent 可观测性把技术团队和非技术团队同时纳入 workflow：前者看性能和错误，后者发现失败模式并标注 trace。<sup>12</sup>

<sup>12</sup>视频画面时间区间：00:12:16.19–00:12:50.00。

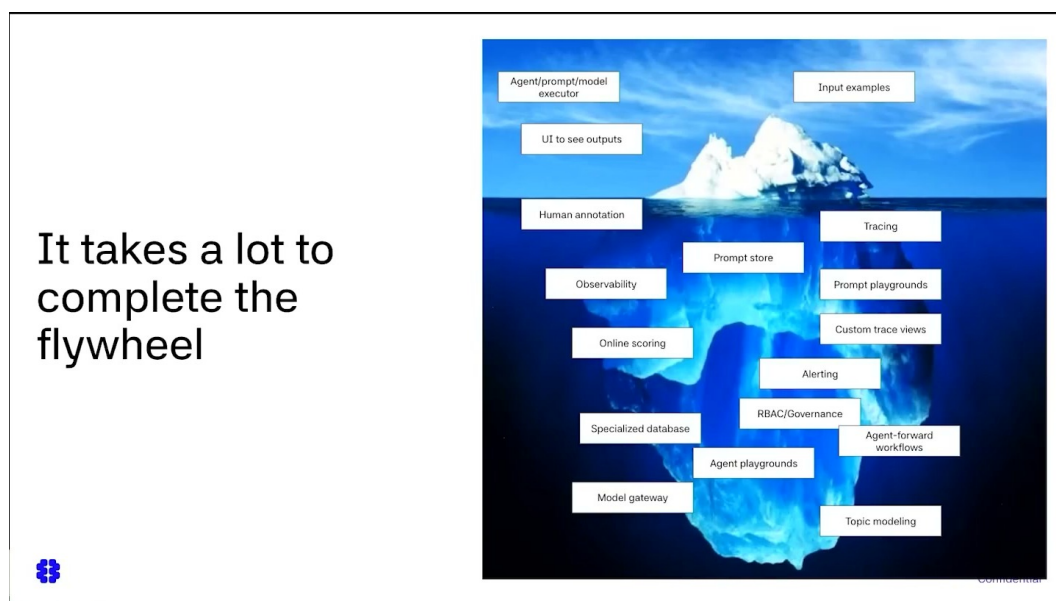


图 13: 完成 Agent 质量飞轮需要一整套水面上下组件：人工标注、可观测性、在线评分、专用数据库和主题建模共同支撑迭代。<sup>13</sup>

## 角色扩展的本质

Agent observability 的对象是 agent 的行为质量。行为质量往往需要业务语境来判断，所以最有判断力的人不总是系统工程师。临床、护理、财富管理、法律、产品等角色会进入 trace 查看、质量判断、prompt 改进和评测设计流程。

## 6.1 传统 observability 的典型用户

传统 observability 的用户画像相对集中。系统工程师关心服务是否在线，平台工程师关心基础设施是否稳定，产品工程师关心接口是否报错、请求是否变慢。它们的共同点是：信号多半可以通过技术指标表达。

例如，一次请求返回 500，系统工程师可以判断它异常；P95 延迟从 300ms 飙到 3s，平台工程师可以判断用户体验变差；某个区域的错误率突然升高，值班人员可以进入排障流程。这些判断不需要临床专家解释医学语义，也不需要律师判断条款风险。

Agent 场景里的质量判断更像阅读一份专业意见书。系统可以告诉人们一次回答耗时 1.2 秒，也可以告诉人们模型调用了三个工具；但“这份回答是否符合护理流程”“这个理财建议是否适合客户风险等级”“这段法律解释是否越界”，需要领域知识。

## 6.2 SME 为什么能直接参与 Agent 改进

Phil 给出的关键机制是 prompt。Prompt 是自然语言贡献通道。过去，非工程角色要改变软件行为，通常需要通过需求文档、工单、会议和工程排期，把知识翻译给开发团队。LLM agent 让一部分行为规则直接写成自然语言：系统提示、评测样例、人工标注理由、失败案例总结，都可以由领域专家参与。

这并不意味着 SME 可以绕过工程治理随意改生产系统。它意味着贡献界面发生了变化。领域专家可以查看 trace，指出 agent 在哪一步误解了用户意图，哪一段回答缺少依据，哪一次工具调用结果被错误解释。他们还可以把这种判断写成 prompt 约束、评分标准或 eval case。

<sup>13</sup>视频画面时间区间：00:13:16.00–00:13:40.00。

### SME 的贡献像“专业审稿”

工程师搭建 agent 的运行框架，领域专家审阅 agent 的行为内容。临床人员看医疗建议是否贴合护理规范，财富顾问看建议是否符合客户画像，律师看法律表达是否越界，产品经理看回答是否符合产品定位。trace 给他们共同的审稿材料，prompt 和 evals 给他们可写入系统的反馈通道。

这一点也解释了为什么 Agent observability 工具需要同时服务后端工程师和领域专家。界面若只暴露内部 span id、底层协议和机器指标，SME 会被挡在门外；系统把输入、上下文、工具调用、回答、评分和标注意见组织成可读材料后，非工程角色才能参与质量闭环。

## 6.3 Prompt：自然语言作为贡献接口

Prompt 在这里已经超出“随手写一句话让模型听话”的小技巧；它是一种组织知识进入系统的接口。它把专业判断从口头讨论转成可版本化、可评测、可回放的文本资产。

可以把 SME 的贡献路径写成：

$$K_{sme} \rightarrow P \rightarrow A(x; P) \rightarrow \tau \rightarrow J_{sme}$$

- $K_{sme}$ ：领域专家掌握的专业知识，例如临床流程、合规边界、法律风险、产品标准。
- $P$ ：prompt 或相关自然语言规则，包括系统提示、任务说明、评分标准。
- $A(x; P)$ ：agent 在输入  $x$  和 prompt  $P$  约束下执行任务。
- $x$ ：用户输入或测试输入。
- $\tau$ ：agent 执行后留下的 trace。
- $J_{sme}$ ：领域专家基于 trace 给出的判断，例如好坏评分、问题说明、修改建议。
- $\rightarrow$ ：信息流向，表示专业知识被写入 prompt，再通过 trace 和判断回流到改进流程。

这个循环的重点是可追溯。领域专家的知识会从会议纪要进入 prompt、eval case、annotation reason 和 scoring rubric。工程师依然负责权限、发布、监控和系统边界，SME 则把业务判断持续注入质量体系。

### 自然语言入口仍需工程治理

Prompt 让领域专家能贡献判断和规则，但生产系统仍然需要版本管理、评审、回滚、权限控制和 eval 验证。自然语言降低了表达门槛，没有取消工程治理。

## 6.4 Observability 与 evals：同一个系统问题的两种运行形态

Phil 在这一段提出一个很重要的统一视角：他们把 observability 和 evals 看成同一个问题。差别在运行方式：evals 是批处理，输入提前已知；observability 是生产实时流，输入来自真实用户，事前未知。

这个观点能把很多概念连起来。Eval 数据像试卷，题目提前准备好，系统批量作答，然后评分；observability 数据像门诊，每个真实用户带着自己的问题进入系统，trace 在现场生成，然后被查看、评分、搜索和分析。两者都需要 trace 存储、查询、评分、标注、聚合和改进闭环。

$$D_{eval} = \{(x_i, y_i)\}_{i=1}^m, \quad D_{obs} = \{\tau_j\}_{j=1}^n$$

- $D_{eval}$ ：离线评测数据集。
- $(x_i, y_i)$ ：第  $i$  个评测样本，包含输入  $x_i$  和可选参考答案或期望判断  $y_i$ 。
- $x_i$ ：预先已知的评测输入。
- $y_i$ ：参考答案、标签、评分标准或期望输出。

- $m$ : 评测样本数量。
- $D_{obs}$ : 生产观测数据集。
- $\tau_j$ : 第  $j$  条生产 trace, 由真实用户交互实时产生。
- $n$ : 生产 trace 数量。

两者的运行差异可以继续写成:

$$Mode(D) = \begin{cases} \text{batch,} & D = D_{eval} \text{ and inputs are known before execution} \\ \text{real-time,} & D = D_{obs} \text{ and inputs appear during production} \end{cases}$$

- $Mode(D)$ : 数据集  $D$  对应的运行模式。
- $D$ : 被处理的数据集。
- $D_{eval}$ : 离线评测数据集。
- $D_{obs}$ : 生产观测数据集。
- batch: 批处理模式, 系统一次性或定期处理一批已知输入。
- real-time: 实时模式, 系统处理生产过程中不断到来的未知输入。
- $\{\dots\}$ : 分段定义, 表示不同条件下取不同模式。

这个统一视角有实际工程后果。存储 trace 的系统、检索 trace 的系统、展示 trace 的系统、让人标注 trace 的系统、把失败案例变成下一轮评测的系统, 应该尽量复用。否则团队会维护两套割裂平台: 一套看线上, 一套跑离线; 线上发现的问题很难进入 evals, evals 中暴露的失败也很难映射回生产 trace。

### 同构问题

Observability 和 evals 的共同底座是“记录 agent 行为, 查询这些行为, 判断这些行为, 把判断反馈到改进流程”。evals 的输入提前摆在桌上, observability 的输入在生产现场到来; 底层能力高度重合。

## 6.5 本章小结

第 6 章的核心是组织边界变化。Agent observability 把可观测性从工程值班室扩展到业务判断现场: clinicians、registered nurses、wealth advisors、lawyers、product managers 和贴近用户的一线角色, 都可能直接阅读 trace, 并把专业判断写回 prompt、标注、评分标准和 eval case。

Prompt 让自然语言成为贡献接口, trace 让 agent 行为变成可审阅材料, evals 与 observability 共用同一类系统能力。离线 evals 处理已知输入的批量样本, 生产 observability 处理实时出现的未知 trace; 两者共同构成 agent 质量改进闭环。

## 7 下一阶段: 从 trace 到主题、意图、情绪与问题发现

### 7.1 看似简单的问题, 背后需要复杂系统托底

演讲主体收束在一个非常朴素的问题上: 既然 Braintrust 已经收集了大量 agent trace, 而且这些 trace 里包含用户输入、模型输出、工具调用、上下文证据和中间推理痕迹, 那么系统能不能直接告诉团队, 用户到底怎样使用这个 agent? 这个问题听起来像一个产品经理随口提出的统计需求, 实际牵涉的系统层次很深。

原因在于, agent trace 更像一批客服录音、工单、医生病历和工程日志混在一起的档案: 每一条都有结构字段, 也有长文本; 每一条都能被人读懂一部分, 却很难靠固定规则穷尽解释。传统监控可以统计“接口 P95 延迟是多少”, 而这里要回答的是“用户来找 agent 做什么、他们感受如何、哪些问题正在浮现”。后者要求系统从单条记录进入群体行为模式。

## 重点提示

本节的关键判断是：agent observability 的下一阶段要超出 UI 中的单条 trace 展示，进一步把大量 trace 转化为可探索的行为地图。地图上的区域可以对应用户意图、使用主题、情绪状态和潜在故障模式。

## 7.2 从 trace 到 embedding：把复杂交互压缩成可计算向量

演讲者提到的第一步是用轻量 LLM 对进入系统的 agent observability traces 做 embedding。直观地说，embedding 是一种语义坐标：原始 trace 可能长达许多页，包含问题、回答、检索证据和工具输出；embedding 把这些内容压缩成一个向量，让相似的交互在向量空间里靠近。

可以把这个过程写成：

$$z_i = f_{\text{embed}}(\tau_i)$$

- $\tau_i$ ：第  $i$  条 agent trace，包含用户输入、模型响应、工具调用、上下文和元数据。
- $f_{\text{embed}}$ ：把 trace 内容映射到向量空间的嵌入模型或轻量 LLM 流程。
- $z_i$ ：第  $i$  条 trace 的向量表示，用于后续相似度计算、聚类 and 主题发现。

这里的压缩服务于机器比较；摘要服务于人工快速阅读。Embedding 让机器能比较大量记录之间的语义距离。例如两个用户的问题表面措辞完全不同，一个说“为什么我的退款没到账”，另一个说“钱退回原卡要多久”，它们在语义空间里应当接近，因为背后的业务意图都指向退款进度。

### 补充知识

向量空间可以理解成一张没有街道名称的城市地图。每个 trace 是地图上的一个点；两个点越近，系统越倾向认为它们语义相似。后续聚类相当于找出地图上的自然街区，而主题建模相当于给这些街区命名。

## 7.3 从 clustering 到 topic modeling：把个体交互组织成群体模式

有了每条 trace 的向量表示，系统就可以进行 clustering。聚类的目的在于从海量生产交互中发现自然形成的簇，避免只给每条 trace 打一个孤立标签：一簇可能是“用户询问发票”，一簇可能是“工具调用失败后用户反复追问”，另一簇可能是“用户想让 agent 解释政策边界”。

一个简化表达是：

$$c_i = \text{Cluster}(z_i), \quad T_k = \{\tau_i \mid c_i = k\}$$

- $z_i$ ：第  $i$  条 trace 的 embedding 向量。
- $\text{Cluster}(\cdot)$ ：聚类算法或聚类流程，用来把相近向量归为同一组。
- $c_i$ ：第  $i$  条 trace 所属的簇编号。
- $k$ ：某一个具体簇的编号。
- $T_k$ ：属于第  $k$  个簇的 trace 集合。

聚类之后才进入 topic modeling，也就是给这些簇提炼可读的主题。主题建模需要超出“Cluster 17”这种机器标签；它需要把一组相似 trace 概括成团队能行动的语言，例如“退订流程不清楚”、“检索到过期资料”、“用户期待人工转接”。这样的主题才可以进入产品、工程和运营的讨论。

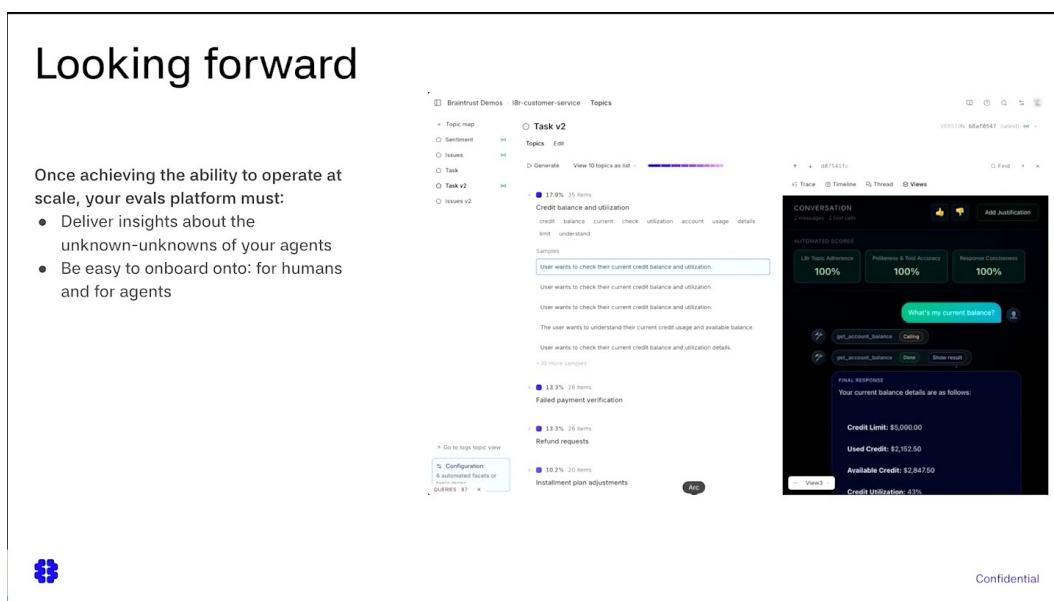


图 14: 下一阶段的 Agent 可观测性会从 trace 中提炼 unknown unknowns: 通过嵌入、聚类 and 主题建模发现用户意图、情绪与问题模式。<sup>14</sup>

### 易错提醒

聚类结果很容易被误读成确定真相。更稳妥的做法是把它看成问题雷达: 它能指示哪里有密集信号, 仍然需要抽样阅读、人工判断和线上验证来确认簇的含义。

## 7.4 intent、sentiment 与 issue discovery: 从发生了什么走向为什么重要

演讲者列出的输出包括用户如何使用 agent、用户意图、互动情绪, 以及他们可能遇到的问题。这四类信息对应四个不同层次。

第一层是使用方式, 即用户把 agent 当成什么工具。比如他们把它当作搜索入口、政策解释器、任务执行器, 还是客户支持替代物。第二层是 intent, 也就是用户真实想完成的任务。意图比关键词更深, 用户写下“这个能退吗”, 背后可能是退款资格判断、订单状态查询, 也可能是政策争议。

第三层是 sentiment, 关注用户在交互中的状态。情绪信号包括正面、负面, 以及困惑、焦虑、受挫、确认和满意等更细粒度信号。第四层是 issue discovery, 也就是从 trace 群体中发现问题: 工具经常拿不到数据、agent 总在某类政策上答得含糊、某个用户路径导致重复追问。

### 重点提示

topic、intent、sentiment 和 issue discovery 的共同价值在于, 它们把 trace 从审计材料提升为产品反馈。单条 trace 解释一次交互, 成簇后的 trace 解释一个用户群体正在遭遇的模式。

## 7.5 缩短从生产问题到实验修复的迭代回路

这一套流程的最终目的, 是缩短从生产问题到实验修复的回路。没有聚类和主题发现时, 团队可能只能依赖少量投诉、人工抽查或某个异常指标来猜测问题。引入 trace embedding 和主题发现之后, 生产中的问题可以更快转化为实验输入: 发现某个主题簇异常增大, 抽样查看代表性 trace, 归纳失败模式, 构造离线 eval 数据集, 再比较 prompt、工具路由或检索策略的修复效果。

<sup>14</sup> 视频画面时间区间: 00:13:46.19–00:15:05.00。

可以把这个闭环抽象成：

$$\Delta t_{\text{loop}} = t_{\text{fix}} - t_{\text{detect}}$$

- $\Delta t_{\text{loop}}$ ：从发现生产问题到完成实验性修复所需的迭代时间。
- $t_{\text{detect}}$ ：系统或团队识别出某类生产问题的时间点。
- $t_{\text{fix}}$ ：团队完成候选修复并能在实验中验证的时间点。

优化目标是让  $\Delta t_{\text{loop}}$  变小。技术上靠 embedding、clustering、topic modeling 和自动化分析减少发现成本；组织上靠产品、工程和领域专家共享同一批 trace，减少解释成本；实验上靠把生产问题转入离线评测，减少试错成本。

#### 补充知识

这里的“loop”更接近机器学习系统的持续校准过程。生产环境提供真实分布，离线实验提供可控对比，两者通过 trace 数据连接起来。

## 7.6 本章小结

第 7 节把 agent observability 从“看见单次交互”推进到“发现群体行为模式”。演讲者保留了一个重要判断：越是简单的问题，背后越可能需要复杂系统支撑。“用户怎样使用我的 agent”这个问题，需要 trace 存储、文本理解、embedding、聚类、主题建模、意图识别、情绪判断和问题发现共同工作。其工程意义很直接：更快看见生产问题，更快把问题转成实验样本，更快验证修复是否有效。

## 8 Q&A：功能可观测性、人类标注与闭环优化

### 8.1 functional observability 与 technical observability 的边界

问答部分首先澄清了一个边界：Braintrust 关注 agent 的 functional observability，也覆盖 technical observability。这里的 functional observability 指团队按自身业务定义来观察 agent 质量，例如回答是否可靠、是否符合政策、是否使用了正确工具、是否满足用户任务。technical observability 则关注 prompts duration、time to first token、cache hit、延迟、token 数等运行指标。

#### 问答片段

听众的问题可以概括为：Braintrust 更像是在观察 agent 的功能表现，还是也能覆盖非功能和技术性能？演讲者的回答是：传统可观测性也能处理一部分技术性能；Braintrust 的核心是按团队定义的 agent 质量进行观察，同时在 trace 应用时自然获得 duration、TTFT、cache hit 等技术信号。

这个回答的实际含义是，agent observability 不需要放弃传统指标。它继承这些指标，然后在上面增加语义质量层。对于一个用户支持 agent 来说，300 毫秒更快的响应没有办法弥补错误退款政策；同样，完全正确的回答如果 30 秒才返回，也会损害用户体验。两类信号要放在同一个观察面板中解释。

#### 重点提示

functional observability 回答“agent 做得好不好”；technical observability 回答“系统运行得顺不顺”。成熟的 agent 平台需要同时回答这两类问题，因为用户体验来自语义质量和技

术性能的共同结果。

## 8.2 Human annotation: 专家判断必须携带理由

第二个实质问题来自听众对“human annotation”的追问。演讲者把场景具体化：一条 trace 进入系统后，产品经理或领域专家需要判断 agent 做得好还是差。关键点包括给分，以及解释为什么这样给分。

这点很重要。单纯的好坏标签像一个温度计读数，只告诉团队热或冷；理由像诊断记录，说明发热可能来自感染、免疫反应或环境变化。对于 agent 质量改进，理由比标签更可迁移，因为理由描述了失败机制。

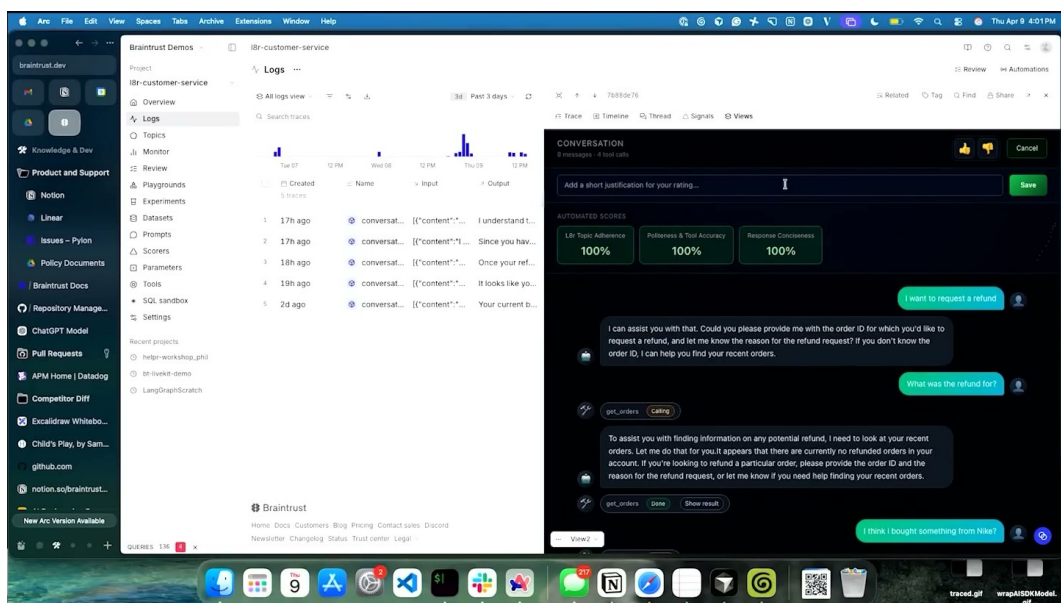


图 15: Q&A 中展示的人工标注界面：领域专家可以对 trace 打分、写 justification，并为后续自动评分函数提供失败模式样本。<sup>15</sup>

### 补充知识

领域专家的价值来自两类知识：一类是贴近用户的情境知识，知道真实用户想要什么；另一类是贴近业务规则的判定知识，知道答案在专业标准下是否成立。human annotation 把这两类知识写回 trace。

## 8.3 从标注理由到 failure modes，再到 scalable scoring functions

演讲者进一步说明，人类标注的理由可以被 LLM 处理，用来生成更可扩展的 scoring functions。流程的重点在于把专家判断结构化，同时保留专家在流程中的判断位置：专家先指出某些回答为什么不合格，系统再从大量理由中归纳 failure modes，最后把这些 failure modes 变成可重复运行的自动评分函数。

这个机制可以表达为：

$$s_j(\tau_i) = g_j(\tau_i, r_i)$$

- $\tau_i$ : 第  $i$  条 agent trace。

<sup>15</sup>视频画面时间区间：00:16:43.96–00:17:10.00。

- $r_i$ : 专家对第  $i$  条 trace 的标注理由，例如“引用了过期政策”或“没有解释工具失败原因”。
- $g_j$ : 针对第  $j$  类 failure mode 的评分函数。
- $s_j(\tau_i)$ : 第  $j$  类评分函数对第  $i$  条 trace 给出的分数或判定。

例如专家多次写下“回答没有基于检索到的合同条款”，系统就可以把它抽象成 groundedness failure mode；专家多次写下“工具返回空结果后 agent 仍然编造答案”，系统就可以抽象成 tool-failure handling failure mode。之后这些 failure modes 能进入自动评分，覆盖比人工抽样大得多的 trace 规模。

#### 易错提醒

自动评分函数的质量取决于上游理由的质量。若人工标注只写“不好”或“错了”，系统很难归纳可执行的 failure mode；若理由写清楚证据、规则和预期行为，自动化才有材料可学。

## 8.4 ClickHouse 历史与文本索引动机

问答中还有一个数据库层面的补充。听众提到 OLAP 和 ClickHouse，演讲者回答说他们过去使用过 ClickHouse，后来迁移到自研数据库。需要谨慎理解这句话：这句话应限定在 Braintrust 当时的特定工作负载中理解。演讲者给出的具体原因是，在当时的工作负载中，他们需要更强的 text-based indexes，而 ClickHouse 当时无法满足他们的特定需求。

这个背景和前文的 Tantivy、full-text indexing 完全衔接。agent trace 的困难同时来自行数多、写入快、分析查询重，以及大量信息埋在自然语言文本里。团队会问“哪些 trace 里出现了 Amazon”，也会问“哪些用户在表达退款受阻”。前者接近关键词检索，后者接近语义检索和主题发现。纯粹按数值列聚合的分析数据库，很难独立覆盖这些文本工作负载。

#### 重点提示

自研数据库的动机来自 trace 的数据形状：实时写入、快速读取、分析查询、全文检索、语义处理和 SQL-like 接口需要在同一系统中协作。数据库选择在这里是工作负载牵引出来的工程结果。

## 8.5 把生产 trace 加入离线数据集

听众还追问了如何与 agentic framework 或离线优化形成闭环。演讲者给出的路径很直接：当生产 trace 进入系统并完成追踪后，可以把它加入 offline dataset，用于后续实验。这样，observability 和 evals 的边界再次被拉近。

在离线 eval 中，输入通常是预先知道的，团队可以批量运行候选 prompt、模型版本、检索配置或工具策略；在生产 observability 中，输入来自真实用户，团队事先并不知道会出现什么。把生产 trace 添加到离线数据集，等于把真实分布中的失败样本带回实验室。

可以把这个数据流写成：

$$D_{\text{offline}}^{(t+1)} = D_{\text{offline}}^{(t)} \cup \{\tau_i \in D_{\text{prod}} \mid h(\tau_i) = 1\}$$

- $D_{\text{offline}}^{(t)}$ : 第  $t$  轮实验时已有的离线数据集。
- $D_{\text{offline}}^{(t+1)}$ : 加入生产样本后的下一轮离线数据集。
- $D_{\text{prod}}$ : 生产环境中收集到的 trace 集合。
- $\tau_i$ : 第  $i$  条生产 trace。
- $h(\tau_i)$ : 样本选择函数，表示某条 trace 是否值得加入离线数据集；取值为 1 时表示选入。
- $\cup$ : 集合并集操作，表示把选中的生产 trace 合并进离线数据集。

这里的  $h(\tau_i)$  可以来自人工挑选；从后续工程设计看，也可以把异常评分、聚类中的代表样本、用户负面情绪、工具失败标记或某个主题簇的高频问题作为候选选择信号。关键是让离线实验跳出旧样本循环，持续吸收生产环境暴露的新问题。

## 8.6 known unknowns 与 unknown unknowns

最后一个实质补充涉及 known unknowns 和 unknown unknowns。known unknowns 是团队已经知道要监控什么，只是还不知道每次运行的结果。例如 groundedness、工具使用正确性、延迟、cache hit、是否违反品牌语气，这些都可以被设计成评分项。unknown unknowns 则指团队事先没有命名的问题模式，例如某个新用户群体把 agent 用在了设计者没预料到的场景，或者某类政策解释在生产中持续触发困惑。

两者需要不同工具。known unknowns 适合在线 scoring：先定义标准，再对 trace 打分。unknown unknowns 适合开放式洞察：先让系统做聚类、主题发现、异常簇提示，再由人类阅读和命名。

### 补充知识

known unknowns 像检查清单，团队知道要检查刹车、胎压和油量；unknown unknowns 像路面勘察，团队要发现地图上还没有标出的塌陷、绕行和拥堵。agent observability 同时需要检查清单和勘察能力。

## 8.7 本章小结

Q&A 把演讲从系统架构推进到实际工作流。第一，agent observability 同时覆盖功能质量和技术性能。第二，human annotation 的核心资产是专家给出的理由，这些理由可以沉淀为 failure modes，并扩展成自动评分函数。第三，Braintrust 离开 ClickHouse 的说法应放在具体历史语境中理解：当时的 agent trace 工作负载需要更强文本索引。第四，生产 trace 可以被加入离线数据集，让真实问题进入实验闭环。第五，known unknowns 适合评分，unknown unknowns 适合开放式发现；两者结合，才接近演讲者所说的下一代 agent observability。

## 9 总结与延伸

### 9.1 演讲者的收束观点

Phil Hetzel 在主体部分给出的路线很清楚：传统可观测性已经能很好回答系统是否在线、延迟是否达标、错误率是否异常；Agent 可观测性继续继承这些底座，同时必须把行为质量、语义证据、工具使用、领域判断和生产反馈纳入同一条链路。Q&A 中的补充进一步说明，Braintrust 把 observability 与 evals 看成同一类系统问题：离线评测提前知道输入，生产观测在真实流量中等待未知输入出现，二者都依赖 trace、评分、检索和分析。

### 9.2 概念压缩：从状态监控到行为诊断

可以把整场演讲压缩成一个公式：

$Agent\ O11y = Technical\ Signals + Semantic\ Evidence + Human\ Judgment + Feedback\ Loop$

- *Technical Signals*: 延迟、错误率、TTFT、token 数、cache hit 等工程信号。
- *Semantic Evidence*: prompt、response、tool output、上下文证据和长文本检索。
- *Human Judgment*: 领域专家对 trace 的好坏判断，以及判断背后的理由。
- *Feedback Loop*: 把生产问题转成离线实验，把人工理由转成可扩展评分函数。

## 核心判断

Agent 可观测性的难点已经超出“采集更多日志”。系统要把一次 agent 交互还原成可查询、可解释、可评分、可被领域专家共同改进的证据链。

### 9.3 给工程团队的落地启示

第一, trace 结构要为未来的问题预留空间。若系统只保存最终回答, 就很难追问 groundedness、工具选择、上下文证据和失败路径。第二, 质量指标要与业务定义绑定。医疗、财富管理、法律、客服等场景对“好回答”的定义不同, 通用延迟指标无法替代领域判断。第三, 人工标注的价值在理由。单纯打分只能告诉团队哪里坏了, 理由才能暴露 failure mode, 并逐步形成自动评分函数。第四, 生产 trace 和离线 eval 数据集应当连通。生产环境发现的问题需要进入实验系统, 实验修复后的版本再回到生产观测中接受检验。

### 9.4 开放问题

- 当 trace 体积极大时, 团队应如何在成本、隐私、实时性和可解释性之间设定保留策略?
- 领域专家参与标注后, 怎样避免评分标准漂移, 并让不同专家的判断保持一致?
- 聚类和主题建模能发现 unknown unknowns, 但这些发现进入产品优先级后, 还需要怎样的人类审查流程?
- Agent 可观测性平台需要处理大量 prompt、用户输入和工具输出, 安全边界与数据治理会成为系统设计的一部分。

### 9.5 拓展阅读

- Braintrust 官方博客可作为进一步了解 agent trace 数据系统与 eval 工作流的入口。
- Apache Lucene 与 Tantivy 的资料适合补充理解全文索引在长文本检索中的作用。